GASGUARD: IoT-Based Liquefied Petroleum Gas Leak Detection and Safety System

---

1. Problem Statement (Real-Life Problem)

LPG leaks pose a silent yet deadly threat that can be difficult to detect by the sense of smell alone. Gas accumulation in enclosed spaces can lead to fires or fatal explosions with little to no warning. In the Philippines, LPG-related incidents remain a recurring cause of residential fires. GasGuard is an IoT-based detection and safety mitigation system designed to actively monitor environments, trigger immediate local and remote alerts, and physically intervene to prevent disaster when gas remains unattended.

Imagine a solo-living senior citizen who finishes cooking, but an old hose fitting begins to leak LPG into the kitchen. As the gas accumulates, the MQ-6 sensor detects the rising PPM. Once it crosses the critical safety threshold, the system executes an automated response:

The buzzer sounds, and the LED turns red.

The LCD displays "DANGER: LEAK DETECTED."

The servo motor physically rotates the LPG valve to the "OFF" position, cutting off the source.

The exhaust fan kicks on to ventilate the kitchen.

Simultaneously, the ESP32 publishes an MQTT payload. The Node.js backend processes this, logs the incident in Firebase, and pushes an immediate critical alert to the Vue.js dashboard, notifying a registered family member remotely.

2. Project Objectives

- Detect LPG leaks with the MQ-6 sensor, trigger the buzzer, and send an alert to the dashboard remotely within 3 seconds of exceeding the safety thresholds.
- Transmit real-time data to the dashboard under 2 seconds of latency using the MQTT protocol.
- Automatically deploy mitigations within 5 seconds of a gas leak detection
- Maintain 99% continuous monitoring uptime to protect the beneficiaries effectively

3. System Overview & Architecture

GasGuard operates as a continuous environmental monitoring and mitigation loop. The ESP32 serves as the core processor/microcontroller, reading real-time gas concentrations via the MQ-6 Gas Sensor and environmental conditions via the DHT22 Temperature & Humidity Sensor.

The gas concentration surpasses a pre-defined threshold, the ESP32 executes local emergency protocols: displaying warnings on the I2C LCD Display, lighting up the LEDs, sounding the Active Piezzo Buzzer, engaging the DC Exhaust Fan to ventilate the area, and triggering the Servo Motor/Valve to cut off the gas supply.

At the same time, the ESP32 publishes this telemetry data to the Node.js backend using the MQTT protocol. The backend processes this data, logs historical events to the Firebase database, and pushes live updates to the Vue frontend dashboard.

Major Components:

- Core Processor: ESP32
- Sensors: MQ-6 Gas Sensor, DHT22 Temperature & Humidity Sensor
- Output: I2C LCD Display, Servo Motor/Valve, DC Exhaust Fan, LEDs, Active Piezzo Buzzer
- Web Stack: Vue (Frontend), Node.js with MQTT (Backend), Firebase (Database).

4. Hardware & Software Components

Hardware Components:

- ESP32
- MQ-6 Gas Sensor
- DHT22 Temperature & Humidity Sensor
- I2C LCD Display
- Servo Motor/Valve
- DC Exhaust Fan
- LEDs
- Active Piezzo Buzzer

Software/Services:

- Vue
- Node.js with MQTT
- Firebase
- Arduino IDE/Platform IO

5. Cloud & Communication Integration

- **Firebase:** Utilized as the database. It is highly suitable for this IoT system because it allows for seamless syncing of sensor logs across connected clients, efficient storage of system states, and reliable push notifications for remote emergency alerts.
- **MQTT (with Node.js):** Acts as the backend messaging protocol. MQTT is the industry standard for embedded hardware because of its lightweight payload overhead, ensuring that urgent gas leak telemetry from the ESP32 reaches the backend broker with minimal latency, even on restricted Wi-Fi networks.

6. Frontend Interaction (Vue.js)

- **Data Visualization:** The interface will feature real-time line charts mapping gas concentrations over time, alongside dynamic gauge components for live temperature and humidity readings from the DHT22.
- **User Action:** The dashboard will include a secure login portal, manual override toggles to remotely force the activation of the DC exhaust fan or shut the gas valve, and an alert log displaying timestamped anomaly events.

7. Development Timeline

- **Week 1–2 (Hardware & Sensors):** Procure hardware components, wire the ESP32 circuit, and calibrate the MQ-6 and DHT22 sensors.
- **Week 3–4 (Actuators & Local Logic):** Integrate the I2C LCD, LEDs, and Buzzer; program the logic for the Servo Motor and DC Exhaust fan to trigger accurately upon crossing defined gas thresholds.
- **Week 5–6 (Backend & Cloud):** Set up the MQTT broker via Node.js, establish the two-way communication pipeline from the ESP32, and integrate Firebase for data persistence.
- **Week 7–8 (Frontend & Testing):** Develop the Vue dashboard, bind live MQTT data to the frontend UI components, and conduct comprehensive end-to-end leak simulation testing.

8. Success Criteria

- The system successfully transmits an alert remotely to the Vue dashboard via MQTT within 2 seconds of the MQ-6 detecting abnormal LPG levels.
- The physical mitigation system (Servo Valve and DC Exhaust Fan) deploys locally with 100% reliability during simulated gas leak scenarios.
- The ESP32 hardware maintains stable sensor telemetry transmission and continuous database logging for an uninterrupted 72-hour stress test.